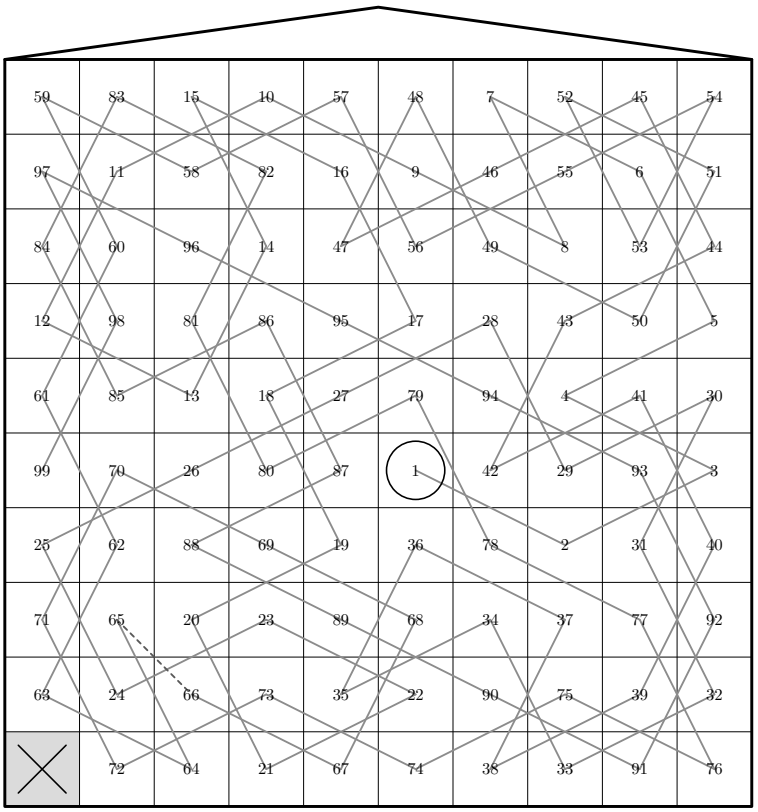
1. The knight's tour. Georges Perec's novel *Life A User's Manual* (*La Vie mode d'emploi*, 1978) is set in an apartment building at 11 rue Simon-Crubellier in Paris. Perec cut the building's facade away like a doll's house, imagining a $10 \times 10 = 100$-cell grid from the cellars to the attic. Each chapter of the novel dwells in one of those 100 cells and tells the story unfolding in that room.

The question is the order in which the narrative moves from room to room. Perec fixed it by the *knight's move* of chess: he chose a knight's tour of the 10×10 board—a path that steps by knight's moves through all 100 cells, never landing on one twice—and let the chapters travel in that order. The first chapter begins at the central landing (6,6).

Yet the novel has 99 chapters, not 100. Partway through the tour Perec deliberately skipped a cell: the cellar at the bottom-left corner (1,10). Borrowing a word from Lucretius, he called this deliberate flaw the *clinamen* (the slight swerve of an atom from its ordained path). Because of it, the move from the 65th chapter to the 66th is not a knight's move but an illegal diagonal step of one cell—the single blemish on an otherwise perfect tour.

Here is a picture. The facade is cut away like a doll's house so that all 100 rooms lie open at once; each cell is stamped with its chapter number, and consecutive chapters are joined by a line. The first chapter is the ringed 1 at the center, and the bottom-left corner—the unvisited clinamen—is left empty with a cross. The dashed line from 65 to 66 is that one illegal, non-knight move.



Perec had a second constraint. He gathered 42 lists of ten items into twenty-one pairs and, by an order-10 Graeco-Latin square, assigned to each chapter a combination of items. If the knight's tour decides *where to write*, this square decides *what to write*. This starred section builds and verifies the *knight's tour*; the square that distributes the material is treated in a later starred section.

2. Here is what the program does. Using GB_BASIC's *Board* it builds the 10×10 knight board, lays Perec's actual chapter order on it, and verifies—by asking the board's own arcs—that the order really is a walk of knight's moves, with the clinamen as its one exception. Finally it prints a grid of chapter numbers and a diagnosis.

The chapter-order data is transcribed from the *sqs* array in `scripts/knights-tour.js`, published by Thomas Guest at `wordaligned.org/knights-tour`. Whether the transcription is faithful the program checks for itself, on the board.

```
package main
import (
    "bufio"
    "fmt"
    "log"
    "os"

    "github.com/sjnam/go-sgb/gbbasic"
)
⟨ Types and data 3 ⟩
func main( ) {
    ⟨ Build the knight board 4 ⟩
    ⟨ Extract the board's adjacency 5 ⟩
    out := bufio.NewWriter(os.Stdout)
    defer out.Flush( )
    ⟨ Verify and print the tour 7 ⟩
    ⟨ Verify the square and print the assignment 17 ⟩
}
```

3. A cell of the board is a **cell**. Here x is the column (1 at the left, 10 at the right) and y the floor (1 at the top, 10 at the bottom). $tour[k]$ is the cell the $(k+1)$th chapter sits in.

⟨ Types and data 3 ⟩ ≡

```
type cell struct { x, y int }
var tour = []cell{
    {6,6},{8,7},{10,6},{8,5},{10,4},{9,2},{7,1},{8,3},{6,2},{4,1},
    {2,2},{1,4},{3,5},{4,3},{3,1},{5,2},{6,4},{4,5},{5,7},{3,8},
    {4,10},{6,9},{4,8},{2,9},{1,7},{3,6},{5,5},{7,4},{8,6},{10,5},
    {9,7},{10,9},{8,10},{7,8},{5,9},{6,7},{8,8},{7,10},{9,9},{10,7},
    {9,5},{7,6},{8,4},{10,3},{9,1},{7,2},{5,3},{6,1},{7,3},{9,4},
    {10,2},{8,1},{9,3},{10,1},{8,2},{6,3},{5,1},{3,2},{1,1},{2,3},
    {1,5},{2,7},{1,9},{3,10},{2,8},{3,9},{5,10},{6,8},{4,7},{2,6},
    {1,8},{2,10},{4,9},{6,10},{8,9},{10,10},{9,8},{7,7},{6,5},{4,6},
    {3,4},{4,2},{2,1},{1,3},{2,5},{4,4},{5,6},{3,7},{5,8},{7,9},
    {9,10},{10,8},{9,6},{7,5},{5,4},{3,3},{1,2},{2,4},{1,6},
}
```

This code is also defined in sections 6, 12, 15.

This code is used in section 2.

4. *Board* builds a board graph from the moves of a generalized chess piece. Its first four arguments are the board's dimensions; a zero dimension is unused, so 10,10,0,0 is a two-dimensional 10×10 board. The fifth argument *piece*=5 is the knight—a knight's move is exactly the one whose Euclidean distance between two cells is $\sqrt{5}$. The last two arguments are wrapping and directedness, both unused here.

⟨ Build the knight board 4 ⟩ ≡

```
g, err := gbbasic.Board(10, 10, 0, 0, 5, 0, false)
if err ≠ Λ {
    log.Fatalf("couldn't build the knight board: %v", err)
}
```

This code is used in section 2.

5. The vertices *Board* makes are named by coordinate `"row.col"`, so the cell at floor y and column x is `"y-1.x-1"` (both counted from 0). Verification needs not the vertices themselves but only whether two cells are neighbors on the board, so we sweep all of the board's arcs into a single set of name pairs. Then one lookup tells whether two cells are joined by a knight's move.

⟨ Extract the board's adjacency 5 ⟩ ≡

```
adj := make(map[[2]string]bool)
for v := range g.AllVertices() {
    for a := range v.AllArcs() {
        adj[[2]string{v.Name, a.Tip.Name}] = true
    }
}
```

This code is used in section 2.

6. This *name* is the bridge from a coordinate to a vertex name: floor y first, column x second, each decremented to count from 0.

⟨ Types and data 3 ⟩ +≡

```
func name(c cell) string { return fmt.Sprintf("%d.%d", c.y − 1, c.x − 1) }
```

7. Now the heart of it. First we sweep Perec's chapter order as a walk on the board, collecting the links that are not neighbors and the cells stepped on twice. Then we pick out the cell, among the 100, that is never stepped on. If Perec's tour is right, the only non-adjacent link is the clinamen, and the only unvisited cell is the cellar corner.

⟨ Verify and print the tour 7 ⟩ ≡

```
fmt.Fprint(out, "PEREC: the knight's tour of Life A User's Manual\n\n")
fmt.Fprintf(out, "  The board's official name is\n  %s\n", g.ID)
fmt.Fprintf(out, "  with %d vertices and %d arcs.\n\n", g.N, g.M)
⟨ Find non-adjacent links and repeated cells 8 ⟩
⟨ Find the unvisited cell 9 ⟩
⟨ Print the chapter-number grid 10 ⟩
⟨ Print the verification result 11 ⟩
```

This code is used in section 2.

8. For each consecutive pair of chapters we ask *adj* whether their cells are neighbors on the board. If not, we record the link in *breaks*. We also record in *chapterAt* the first chapter to step on each cell, so a cell stepped on twice shows up at once.

⟨ Find non-adjacent links and repeated cells 8 ⟩ ≡

```
chapterAt := make(map[string]int)
var breaks [][2]int   // (earlier chapter, later chapter) of a non-adjacent link
var repeats int
for k, c := range tour {
    if _, seen := chapterAt[name(c)]; seen {
        repeats++
    }
    chapterAt[name(c)] = k + 1
    if k > 0 ∧ ¬adj[[2]string{name(tour[k − 1]), name(c)}] {
        breaks = append(breaks, [2]int{k, k + 1})
    }
}
```

This code is used in section 7.

9. We walk all 100 cells of the board and gather those not in *chapterAt*, forming the coordinates by *Board*'s naming rule from floor y and column x.

⟨ Find the unvisited cell 9 ⟩ ≡

```
var missing []cell
for y := 1; y ≤ 10; y++ {
    for x := 1; x ≤ 10; x++ {
        if _, seen := chapterAt[name(cell{x, y})]; ¬seen {
            missing = append(missing, cell{x, y})
        }
    }
}
```

This code is used in section 7.

10. The chapter-number grid is a copy of the novel's facade. From the top-left (1,1) to the bottom-right (10,10), each cell is stamped with the chapter that treats that room. The unvisited cellar corner is left blank with three dots.

⟨ Print the chapter-number grid 10 ⟩ ≡

```
fmt.Fprint(out, "␣␣Chapter-number␣grid␣(top-left␣is␣(1,1),␣top␣row␣is␣the␣attic):\n\n")
for y := 1; y ≤ 10; y++ {
    fmt.Fprint(out, "␣␣␣")
    for x := 1; x ≤ 10; x++ {
        if ch, seen := chapterAt[name(cell{x, y})]; seen {
            fmt.Fprintf(out, "␣%3d", ch)
        } else {
            fmt.Fprint(out, "␣...")
        }
    }
    fmt.Fprint(out, "\n")
}
fmt.Fprint(out, "\n")
```

This code is used in section 7.

11. Finally we put the verification into words a reader can follow. For each non-adjacent link we tell between which chapters, and to which cell, it strayed, and whether the stray is a single diagonal step. Perec's clinamen is exactly that one move from the 65th chapter to the 66th.

⟨ Print the verification result 11 ⟩ ≡

```
fmt.Fprintf(out, "␣␣chapters:␣%d,␣repeated␣cells:␣%d\n", len(tour), repeats)
for _, m := range missing {
    fmt.Fprintf(out, "␣␣unvisited␣cell:␣(%d,%d)␣␣<-␣clinamen\n", m.x, m.y)
}
for _, b := range breaks {
    p, q := tour[b[0] − 1], tour[b[1] − 1]
    dx, dy := abs(p.x − q.x), abs(p.y − q.y)
    fmt.Fprintf(out,
        "␣␣non-knight␣move:␣ch.%d␣(%d,%d)␣->␣ch.%d␣(%d,%d),␣offset␣(%d,%d)\n",
        b[0], p.x, p.y, b[1], q.x, q.y, dx, dy)
}
```

This code is used in section 7.

12. A small hand for the absolute value of a coordinate difference.

⟨ Types and data 3 ⟩ +≡

```
func abs(n int) int {
    if n < 0 {
        return −n
    }
    return n
}
```

13. Run the program and it prints this for the tour: the board's official name and its vertex and arc counts, the chapter-number grid modeled on the novel's facade, and then the verification result.

```
PEREC: the knight's tour of Life A User's Manual

  The board's official name is
  board(10,10,0,0,5,0,0)
  with 100 vertices and 576 arcs.

  Chapter-number grid (top-left is (1,1), top row is the attic):

     59  83  15  10  57  48   7  52  45  54
     97  11  58  82  16   9  46  55   6  51
     84  60  96  14  47  56  49   8  53  44
     12  98  81  86  95  17  28  43  50   5
     61  85  13  18  27  79  94   4  41  30
     99  70  26  80  87   1  42  29  93   3
     25  62  88  69  19  36  78   2  31  40
     71  65  20  23  89  68  34  37  77  92
     63  24  66  73  35  22  90  75  39  32
    ...  72  64  21  67  74  38  33  91  76

  chapters: 99, repeated cells: 0
  unvisited cell: (1,10)  <- clinamen
  non-knight move: ch.65 (2,8) -> ch.66 (3,9), offset (1,1)
```

14. A Graeco-Latin square. If the knight's tour decides *where to write*, *what to write* is decided by Perec's second constraint. As we said, he gathered 42 lists of material into twenty-one pairs and, by a 10×10 Graeco-Latin square, assigned to each chapter its combination of material. But why order 10? Behind that number lies two centuries of mathematical drama, and Perec chose this square knowing the story.

A Graeco-Latin square is two Latin squares laid one over the other, each cell holding a pair of symbols, so that all the pairs are distinct. The story begins in 1782 with Euler's *problem of the 36 officers*. Can thirty-six officers, one of each of six ranks and six regiments, be drawn up in a 6×6 array so that every row and every column shows each rank once and each regiment once? That is precisely an order-6 Graeco-Latin square. Euler could not build one however he tried, and finally conjectured that no such square exists whenever the order is of the form $4k + 2$ (that is, $2, 6, 10, 14, \ldots$).

Euler's conjecture was half right and half wrong. In 1900 Gaston Tarry proved, by counting every case by hand, that the order-6 square really is impossible—so for 6 Euler was right. But not beyond it. At the April 1959 meeting of the American Mathematical Society in New York, Bose, Shrikhande and Parker announced that for every order of the form $4k + 2$ except 2 and 6—that is, $10, 14, 18, \ldots$—a Graeco-Latin square exists. Parker found an order-10 square in about an hour's search on a UNIVAC 1206 military computer, one of the earliest combinatorial problems solved on a digital computer. The three were nicknamed *Euler's spoilers*, and that November the cover of *Scientific American* carried their order-10 square in full colour.

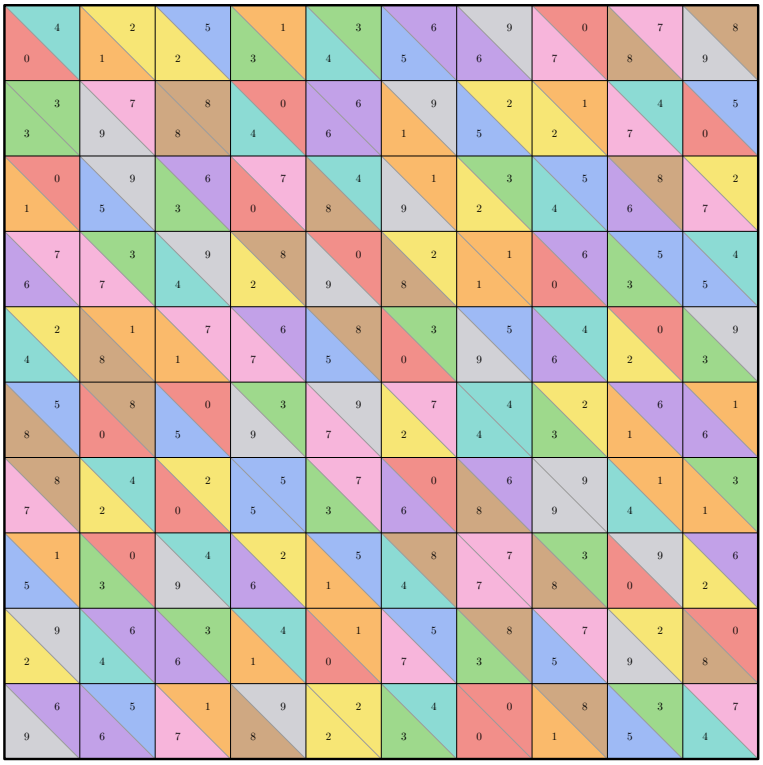
So order 10 is no ordinary number. The very order Euler declared impossible, overturned only two centuries later—the square whose existence had just been proved—is the one Perec took for the skeleton of his novel. It is a choice worthy of a member of Oulipo, who prized the beauty of constraint above all. Now it is time to build the square in earnest and, as Perec did, assign the material to each chapter.

15. For an order that is odd or a prime power, a square is built by a simple formula. For a prime p, say, $L1(i,j) = (i+j) \bmod p$ and $L2(i,j) = (i+2j) \bmod p$ are already two orthogonal Latin squares. But 10 is even and the formula breaks—which is just what fooled Euler and set Parker to his computer. So we do exactly as Parker did: we run a search that builds a random Latin square and hunts for its orthogonal mate, obtain one square (a matter of seconds on today's machines), and set the result down here as *square*. Each cell is a two-component [2]**int**{a, b}.

⟨ Types and data 3 ⟩ +≡

```
var square = [10][10][2]int{
    {{4,0},{2,1},{5,2},{1,3},{3,4},{6,5},{9,6},{0,7},{7,8},{8,9}},
    {{3,3},{7,9},{8,8},{0,4},{6,6},{9,1},{2,5},{1,2},{4,7},{5,0}},
    {{0,1},{9,5},{6,3},{7,0},{4,8},{1,9},{3,2},{5,4},{8,6},{2,7}},
    {{7,6},{3,7},{9,4},{8,2},{0,9},{2,8},{1,1},{6,0},{5,3},{4,5}},
    {{2,4},{1,8},{7,1},{6,7},{8,5},{3,0},{5,9},{4,6},{0,2},{9,3}},
    {{5,8},{8,0},{0,5},{3,9},{9,7},{7,2},{4,4},{2,3},{6,1},{1,6}},
    {{8,7},{4,2},{2,0},{5,5},{7,3},{0,6},{6,8},{9,9},{1,4},{3,1}},
    {{1,5},{0,3},{4,9},{2,6},{5,1},{8,4},{7,7},{3,8},{9,0},{6,2}},
    {{9,2},{6,4},{3,6},{4,1},{1,0},{5,7},{8,3},{7,5},{2,9},{0,8}},
    {{6,9},{5,6},{1,7},{9,8},{2,2},{4,3},{0,0},{8,1},{3,5},{7,4}},
}
```

16. Here it is in colour. Like that famous *Scientific American* cover, each cell is split on its diagonal, the upper triangle coloured by the first component and the lower by the second. Because each component is Latin, the ten colours each appear once in every row and column; because the two are orthogonal, no upper-lower pair of colours repeats across the hundred cells. That is the visible proof that this picture is a genuine Graeco-Latin square.



17. We do not take on trust that the square we set down is really Graeco-Latin; we check, in the same spirit as for the knight's tour. Three things: is the first component Latin, is the second Latin, and are all 100 pairs distinct (which is exactly the orthogonality of the two components). If all three hold, we hold in our hands the very thing Euler said could not exist.

⟨ Verify the square and print the assignment 17 ⟩ ≡

```
fmt.Fprint(out, "\nPEREC: an order-10 Graeco-Latin square\n\n")
⟨ Check that the square is Graeco-Latin 18 ⟩
⟨ Print the square as a grid 19 ⟩
⟨ Assign one couple of lists to chapters 20 ⟩
```

This code is used in section 2.

18. For a component to be Latin means that in each of the ten rows and columns the symbols 0 through 9 each appear once, so it is enough to OR ten bits and see whether they make 1023. Orthogonality we check by putting the 100 pairs into a set and seeing that its size is 100.

⟨ Check that the square is Graeco-Latin 18 ⟩ ≡

```
latinA, latinB := true, true
for i := 0; i < 10; i++ {
    var rA, cA, rB, cB int
    for j := 0; j < 10; j++ {
        rA |= 1 << square[i][j][0]
        cA |= 1 << square[j][i][0]
        rB |= 1 << square[i][j][1]
        cB |= 1 << square[j][i][1]
    }
    if rA ≠ 1023 ∨ cA ≠ 1023 {
        latinA = false
    }
    if rB ≠ 1023 ∨ cB ≠ 1023 {
        latinB = false
    }
}
seen := make(map[[2]int]bool)
for i := 0; i < 10; i++ {
    for j := 0; j < 10; j++ {
        seen[square[i][j]] = true
    }
}
fmt.Fprintf(out, "␣␣Are␣both␣components␣Latin␣squares?␣␣A:␣%v,␣B:␣%v\n", latinA, latinB)
fmt.Fprintf(out, "␣␣Are␣all␣100␣pairs␣distinct␣(orthogonal)?␣␣%v␣(%d␣distinct␣pairs)\n",
    len(seen) ≡ 100, len(seen))
if latinA ∧ latinB ∧ len(seen) ≡ 100 {
    fmt.Fprint(out, "␣␣=>␣here␣is␣the␣order-10␣square␣Euler␣said␣could␣not␣exist.\n\n")
}
```

This code is used in section 17.

19. To show the square itself, we print each cell as two digits ab. That these hundred pairs run over 00 through 99 exactly once each is guaranteed by the check above.

⟨ Print the square as a grid 19 ⟩ ≡

```
fmt.Fprint(out, "␣␣The␣square␣(each␣cell␣is␣two␣components␣ab):\n\n")
for i := 0; i < 10; i++ {
    fmt.Fprint(out, "␣␣␣")
    for j := 0; j < 10; j++ {
        fmt.Fprintf(out, "␣%d%d", square[i][j][0], square[i][j][1])
    }
    fmt.Fprint(out, "\n")
}
fmt.Fprint(out, "\n")
```

This code is used in section 17.

20. Now the two constraints meet. Each cell (x, y) of the building is given a pair (a, b) by the square. Perec bundled his lists of material ten at a time into a couple, and put the ath item of the one list and the bth of the other into the chapter of that room. Because the square is orthogonal, the hundred cells show a hundred combinations exactly once each—any pairing of the two lists meets in the novel exactly once.

Perec kept twenty-one such couples, but his lists in full and their cell-by-cell assignment fill the vast material of his working notebook (the *cahier des charges*). Here, only to show the structure, we use one illustrative couple: ten animals and ten colours.

```
⟨ Assign one couple of lists to chapters 20 ⟩ ≡
    animals := [10]string{"cat", "dog", "horse", "fox", "bear", "deer", "rabbit", "wolf", "hawk",
        "mouse"}
    colours := [10]string{"red", "orange", "yellow", "green", "blue", "indigo", "violet", "black",
        "white", "gray"}
    fmt.Fprint(out, "␣␣One␣couple␣(animal,␣colour)␣assigned␣to␣chapters␣(first␣eight):\n\n")
    used := make(map[[2]int]bool)
    for k, c := range tour {
        p := square[c.y − 1][c.x − 1]
        used[p] = true
        if k < 8 {
            fmt.Fprintf(out, "␣␣ch.%2d␣(%d,%d):␣%s,␣%s\n",
                k + 1, c.x, c.y, animals[p[0]], colours[p[1]])
        }
    }
    ⟨ Show the missing combination is the clinamen's 21 ⟩
```

This code is used in section 17.

21. The ninety-nine chapters use ninety-nine distinct combinations. Exactly one of the hundred is missing—the combination the unvisited clinamen cell $(1, 10)$ would have held. The blemish in the tour that fixes the order has taken away one of the things to write as well.

```
⟨ Show the missing combination is the clinamen's 21 ⟩ ≡
    fmt.Fprintf(out, "\n␣␣combinations␣used:␣%d␣(of␣100)\n", len(used))
    q := square[10 − 1][1 − 1]// clinamen cell (1,10)
    fmt.Fprintf(out, "␣␣missing␣combination:␣%s,␣%s␣␣<-␣the␣share␣of␣clinamen␣cell␣(1,10)\n",
        animals[q[0]], colours[q[1]])
```

This code is used in section 20.

22. The output for the square continues like this: the verification result, the square itself, and one illustrative couple assigned to chapters. Ninety-nine combinations are used, and the last two lines confirm that the one share of the clinamen cell is missing.

```
PEREC: an order-10 Graeco-Latin square

  Are both components Latin squares?  A: true, B: true
  Are all 100 pairs distinct (orthogonal)?  true (100 distinct pairs)
  => here is the order-10 square Euler said could not exist.

  The square (each cell is two components ab):

    40 21 52 13 34 65 96 07 78 89
    33 79 88 04 66 91 25 12 47 50
    01 95 63 70 48 19 32 54 86 27
    76 37 94 82 09 28 11 60 53 45
    24 18 71 67 85 30 59 46 02 93
    58 80 05 39 97 72 44 23 61 16
    87 42 20 55 73 06 68 99 14 31
    15 03 49 26 51 84 77 38 90 62
    92 64 36 41 10 57 83 75 29 08
    69 56 17 98 22 43 00 81 35 74

  One couple (animal, colour) assigned to chapters (first eight):

  ch. 1 (6,6): wolf, yellow
  ch. 2 (8,7): mouse, gray
  ch. 3 (10,6): dog, violet
  ch. 4 (8,5): bear, violet
  ch. 5 (10,4): bear, indigo
  ch. 6 (9,2): bear, black
  ch. 7 (7,1): mouse, violet
  ch. 8 (8,3): deer, blue

  combinations used: 99 (of 100)
  missing combination: rabbit, gray  <- the share of clinamen cell (1,10)
```

23. Index.

⟨ Assign one couple of lists to chapters 20 ⟩ Used in section 17
⟨ Build the knight board 4 ⟩ Used in section 2
⟨ Check that the square is Graeco-Latin 18 ⟩ Used in section 17
⟨ Extract the board's adjacency 5 ⟩ Used in section 2
⟨ Find non-adjacent links and repeated cells 8 ⟩ Used in section 7
⟨ Find the unvisited cell 9 ⟩ Used in section 7
⟨ Print the chapter-number grid 10 ⟩ Used in section 7
⟨ Print the square as a grid 19 ⟩ Used in section 17
⟨ Print the verification result 11 ⟩ Used in section 7
⟨ Show the missing combination is the clinamen's 21 ⟩ Used in section 20
⟨ Types and data 3 ⟩ Used in section 2
⟨ Verify and print the tour 7 ⟩ Used in section 2
⟨ Verify the square and print the assignment 17 ⟩ Used in section 2

PEREC